НАО «КАРАГАНДИНСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ АБЫЛКАСА САГИНОВА»

Кафедра Кибербезопасности и искусственного интеллекта

Лабораторная работа № 4

по дисциплине «Безопасность Баз данных»
Тема:«Маскировка данных в Microsoft SQL Server»
Специальность: 6В06301 – «Системы информационной безопасности»

Принял: ст.преп.
Жакина М.М.
Выполнил:
СИБ 23-7 Гутникевич Е.А.

Караганда 2026

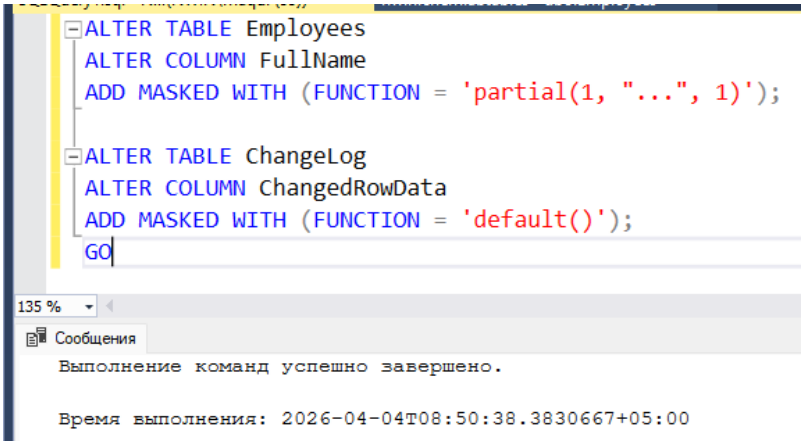
Задания

Используя вашу существующую базу данных и таблицы из индивидуального варианта, выполнить все действия, приведенные в примерах выше. Необходимо:

1. Выбрать таблицы, содержащие чувствительные данные.
Чувствительные данные:
- Employees.FullName → ФИО сотрудников
- ChangeLog.ChangedRowData → JSON с историей изменений

2. Добавить маскировку к существующим столбцам.

```
ALTER TABLE Employees
ALTER COLUMN FullName
ADD MASKED WITH (FUNCTION = 'partial(1, "...", 1)');
ALTER TABLE ChangeLog
ALTER COLUMN ChangedRowData
ADD MASKED WITH (FUNCTION = 'default()');
GO
```



-- Дополнительные маски для тестирования разных типов (можно будет убрать)

```
ALTER TABLE Departments
ALTER COLUMN DepartmentName
ADD MASKED WITH (FUNCTION = 'default()');
ALTER TABLE Furnaces
ALTER COLUMN FurnaceName
ADD MASKED WITH (FUNCTION = 'default()');
GO
```

```sql
-- Дополнительные маски для тестирования разных типов (можно будет убрать)
ALTER TABLE Departments
ALTER COLUMN DepartmentName
ADD MASKED WITH (FUNCTION = 'default()');

ALTER TABLE Furnaces
ALTER COLUMN FurnaceName
ADD MASKED WITH (FUNCTION = 'default()');
GO
```

135 %

Сообщения

```
Выполнение команд успешно завершено.

Время выполнения: 2026-04-04T08:52:00.1043826+05:00
```

3. Проверить отображение данных под учетной записью обычного пользователя.

```sql
EXECUTE AS USER = 'Operator';
GO
SELECT TOP 5
   EmployeeId, FullName, DepartmentId
FROM Employees;
SELECT TOP 5
   DepartmentId, DepartmentName
FROM Departments;
SELECT TOP 5
   FurnaceId, FurnaceName
FROM Furnaces;
SELECT TOP 8
   LogId, ActionType, TableName, ChangedRowData, ChangeDateTime
FROM ChangeLog
ORDER BY LogId DESC;
REVERT;
GO
```

SQLQuery1.sql - N...(NWIN\maqui (55))* | nwin.chemlabtables - dbo.Employees

```sql
EXECUTE AS USER = 'Operator';
GO

SELECT TOP 5
    EmployeeId, FullName, DepartmentId
FROM Employees;

SELECT TOP 5
    DepartmentId, DepartmentName
FROM Departments;

SELECT TOP 5
    FurnaceId, FurnaceName
FROM Furnaces;

SELECT TOP 8
    LogId, ActionType, TableName, ChangedRowData, ChangeDateTime
FROM ChangeLog
ORDER BY LogId DESC;

REVERT;
GO
```

135 %

Результаты | Сообщения

	EmployeeId	FullName	DepartmentId
1	1	J...h	1
2	2	A...n	2
3	3	M...n	3
4	4	E...s	4
5	5	W...n	5

	DepartmentId	DepartmentName
1	9	xxxx
2	6	xxxx
3	8	xxxx
4	10	xxxx
5	5	xxxx

	FurnaceId	FurnaceName
1	1	xxxx
2	2	xxxx
3	3	xxxx
4	4	xxxx
5	5	xxxx

Доступ в Changelog Оператору запрещён

4. Создать отдельного пользователя без права UNMASK.

```sql
USE master;
GO
IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name = 'MaskTestUser')
```

CREATE LOGIN MaskTestUser WITH PASSWORD = 'StrongPass123!', CHECK_EXPIRATION = OFF;

USE chemlabtables;

GO

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'MaskTestUser')

CREATE USER MaskTestUser FROM LOGIN MaskTestUser;

-- Права на чтение чувствительных таблиц

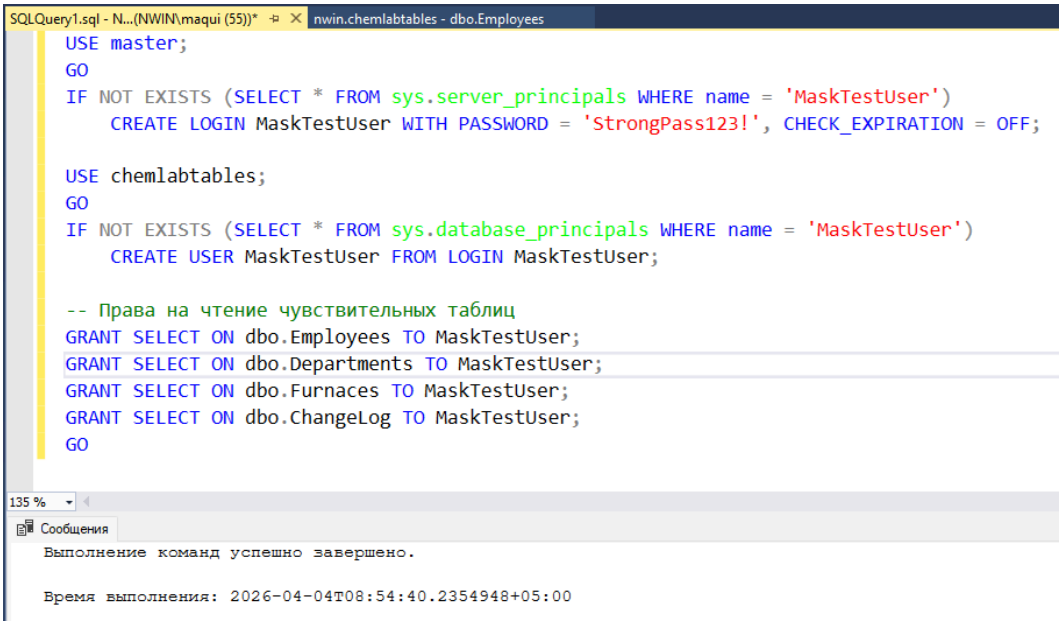
GRANT SELECT ON dbo.Employees TO MaskTestUser;

GRANT SELECT ON dbo.Departments TO MaskTestUser;

GRANT SELECT ON dbo.Furnaces TO MaskTestUser;

GRANT SELECT ON dbo.ChangeLog TO MaskTestUser;

GO



5. Проверить работу функций маскировки для различных типов данных.

EXECUTE AS USER = 'MaskTestUser';

GO

PRINT '=== Проверка маскировки FullName (partial) ===';

SELECT TOP 5 EmployeeId, FullName FROM Employees;

PRINT '=== Проверка маскировки ChangedRowData (default + NVARCHAR(MAX)) ===';

SELECT TOP 5 LogId, ChangedRowData FROM ChangeLog ORDER BY LogId DESC;

PRINT '=== Проверка default() на строковых полях ===';

SELECT TOP 3 DepartmentId, DepartmentName FROM Departments;

SELECT TOP 3 FurnaceId, FurnaceName FROM Furnaces;

REVERT;

GO

```sql
EXECUTE AS USER = 'MaskTestUser';
GO

PRINT '=== Проверка маскировки FullName (partial) ===';
SELECT TOP 5 EmployeeId, FullName FROM Employees;

PRINT '=== Проверка маскировки ChangedRowData (default + NVARCHAR(MAX)) ===';
SELECT TOP 5 LogId, ChangedRowData FROM ChangeLog ORDER BY LogId DESC;

PRINT '=== Проверка default() на строковых полях ===';
SELECT TOP 3 DepartmentId, DepartmentName FROM Departments;
SELECT TOP 3 FurnaceId, FurnaceName FROM Furnaces;

REVERT;
GO
```

135 %

Результаты | Сообщения

	EmployeeId	FullName
1	1	J...h
2	2	A...n
3	3	M...n
4	4	E...s
5	5	W...n

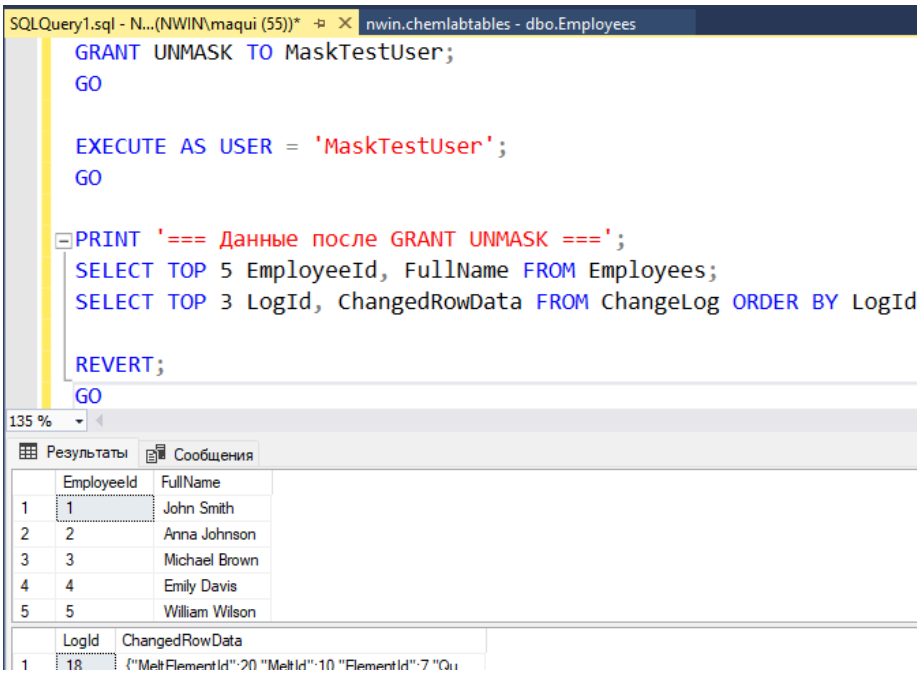
	LogId	ChangedRowData
1	18	xxxx
2	17	xxxx
3	16	xxxx
4	15	xxxx
5	14	xxxx

	DepartmentId	DepartmentName
1	9	xxxx
2	6	xxxx
3	8	xxxx

	FurnaceId	FurnaceName
1	1	xxxx
2	2	xxxx
3	3	xxxx

6. Выдать пользователю право UNMASK и проверить отображение реальных значений.

```sql
GRANT UNMASK TO MaskTestUser;
GO
EXECUTE AS USER = 'MaskTestUser';
GO
PRINT '=== Данные после GRANT UNMASK ===';
SELECT TOP 5 EmployeeId, FullName FROM Employees;
SELECT TOP 3 LogId, ChangedRowData FROM ChangeLog ORDER BY LogId DESC;
REVERT;
GO
```



7. Отозвать право UNMASK.

```
REVOKE UNMASK FROM MaskTestUser;
GO
-- Проверка после отзыва
EXECUTE AS USER = 'MaskTestUser';
GO
SELECT TOP 3 EmployeeId, FullName FROM Employees;
REVERT;
GO
```

```sql
REVOKE UNMASK FROM MaskTestUser;
GO

-- Проверка после отзыва
EXECUTE AS USER = 'MaskTestUser';
GO
SELECT TOP 3 EmployeeId, FullName FROM Employees;
REVERT;
GO
```

135 %

Результаты | Сообщения

	EmployeeId	FullName
1	1	J...h
2	2	A...n
3	3	M...n

8. Удалить маскировку с выбранных столбцов.
ALTER TABLE Departments
ALTER COLUMN DepartmentName
DROP MASKED;
ALTER TABLE Furnaces
ALTER COLUMN FurnaceName
DROP MASKED;
GO

```sql
ALTER TABLE Departments
ALTER COLUMN DepartmentName
DROP MASKED;

ALTER TABLE Furnaces
ALTER COLUMN FurnaceName
DROP MASKED;
GO
```

135 %

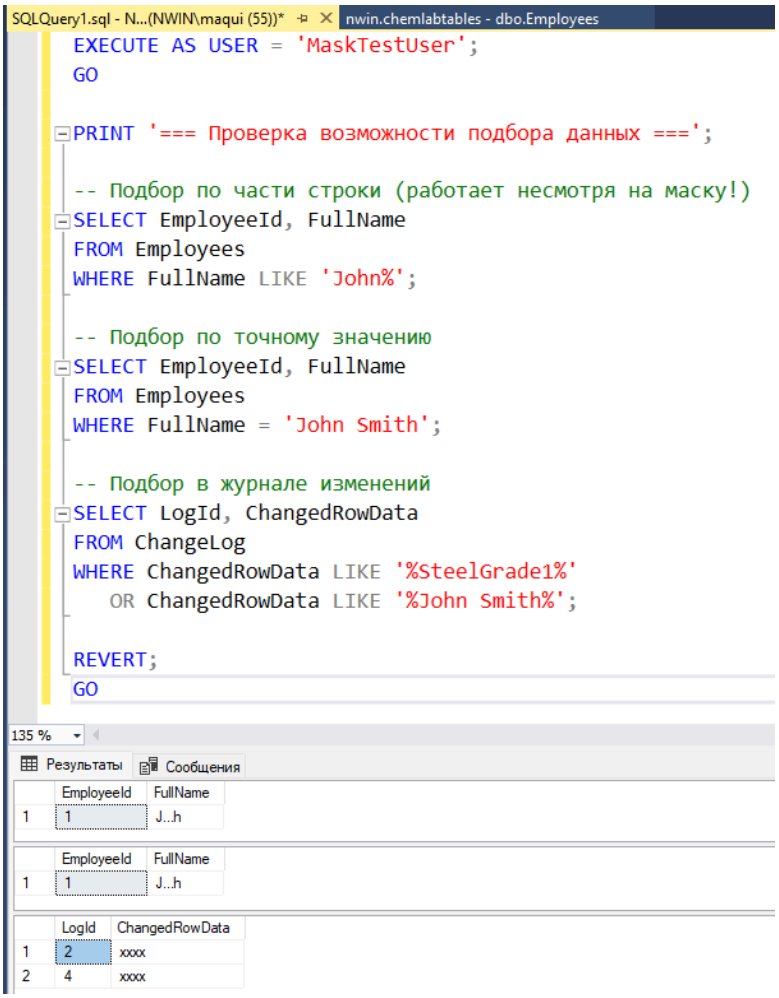
Сообщения

```
Выполнение команд успешно завершено.

Время выполнения: 2026-04-04T08:58:43.2319370+05:00
```

9. Проверить возможность подбора данных через SELECT, WHERE, LIKE и другие условия. Названия таблиц, столбцов и пользователей должны соответствовать структуре вашей базы данных и индивидуальному варианту.

```
EXECUTE AS USER = 'MaskTestUser';
GO
PRINT '=== Проверка возможности подбора данных ===';
-- Подбор по части строки (работает несмотря на маску!)
SELECT EmployeeId, FullName
FROM Employees
WHERE FullName LIKE 'John%';
-- Подбор по точному значению
SELECT EmployeeId, FullName
FROM Employees
WHERE FullName = 'John Smith';
-- Подбор в журнале изменений
SELECT LogId, ChangedRowData
FROM ChangeLog
WHERE ChangedRowData LIKE '%SteelGrade1%'
  OR ChangedRowData LIKE '%John Smith%';
REVERT;
GO
```



Контрольные вопросы

1. Что такое Dynamic Data Masking?

Dynamic Data Masking (DDM) — это функция безопасности SQL Server, которая динамически скрывает (маскирует) чувствительные данные в результате запроса для непривилегированных пользователей, не изменяя сами данные в таблице.

Маскировка происходит только на уровне отображения (при выполнении SELECT). Пользователи с правом UNMASK видят реальные данные, а обычные пользователи — замаскированные.

В вашей БД мы применяли DDM к Employees.FullName и ChangeLog.ChangedRowData.

2. Чем маскировка отличается от шифрования?

Характеристика	Dynamic Data Masking	Шифрование (Encryption)
Изменение данных	Данные в таблице не меняются	Данные хранятся в зашифрованном виде
Восстановление оригинала	Невозможно (маскировка необратима)	Возможно с помощью ключа
Уровень применения	На уровне запроса (при SELECT)	На уровне хранения и передачи
Производительность	Минимальное влияние	Более высокая нагрузка (особенно при частом доступе)
Защита от администратора	Слабая (db_owner видит данные)	Сильная (даже администратор не видит без ключа)
Основное назначение	Скрытие данных для разработчиков/аналитиков	Защита данных при утечке или компрометации

Вывод: Маскировка — это удобный способ ограничения видимости, а шифрование — настоящая защита данных.

3. Какие функции маскировки существуют?

В SQL Server доступны 4 основные функции маскировки:

default() — Полная маскировка по умолчанию в зависимости от типа данных (строка → XXXX, число → 0, дата → 1900-01-01, etc.)

partial(prefix, padding, suffix) — Частичная маскировка с указанием префикса, заполнителя и суффикса Пример: partial(1, "...", 1) → J...h

email() — Специальная маскировка для email-адресов Пример: dinesh@somewhere.edu → dXXXX@XXXX.com

random(start, end) — Замена числового значения на случайное число в заданном диапазоне Пример: random(1000, 5000) для зарплаты или возраста

4. Для чего используется функция partial()?

Функция partial() используется, когда нужно частично показать реальные данные, чтобы они оставались узнаваемыми, но не полностью раскрытыми.

Она позволяет указать:

Сколько символов показывать в начале строки (prefix)

Какой текст использовать для заполнения середины (padding)

Сколько символов показывать в конце строки (suffix)

5. Для чего используется функция email()?

Функция email() предназначена специально для маскировки адресов электронной почты.

Она показывает первую букву локальной части и домен .com, а всё остальное скрывает.

Пример:

Реальный: anna.johnson@company.ru

Замаскированный: aXXX@XXXX.com

Используется, когда нужно сохранить формат email, но скрыть реальный адрес.

6. Для чего нужно право UNMASK?

Право UNMASK позволяет пользователю видеть реальные (немаскированные) данные в столбцах, к которым применена маскировка.

Без UNMASK → пользователь видит замаскированные значения (XXXX, J...h и т.д.)

С UNMASK → пользователь видит оригинальные данные

7. Как добавить маскировку к существующему столбцу?

Используется команда ALTER TABLE ... ALTER COLUMN ... ADD MASKED

8. Как удалить маскировку?

Используется команда DROP MASKED

9. Почему маскировка не является полноценной защитой?

Потому что Dynamic Data Masking — это не механизм безопасности, а механизм сокрытия данных на уровне представления. Основные причины:

Маскировка применяется только при SELECT — данные в таблице хранятся в открытом виде.

Не защищает от подбора через WHERE, LIKE, GROUP BY, JOIN и другие условия (можно угадать значение перебором).

Пользователи с правами db_owner, ALTER ANY MASK или UNMASK видят всё.

Не защищает от прямого доступа к бэкапам, файлов базы или репликации.

Можно обойти через создание представлений, хранимых процедур или UNION.

10. Какими способами можно попытаться угадать замаскированные данные?

Несмотря на маскировку, данные можно попытаться угадать следующими способами:

Условия в WHERE и LIKE — WHERE FullName LIKE 'John%' или WHERE ChangedRowData LIKE '%SteelGrade1%'

Агрегатные функции и группировка (GROUP BY, HAVING, COUNT, SUM)

JOIN с другими таблицами, где данные не замаскированы

Создание представлений или хранимых процедур, которые обходит маску

Логический перебор известных значений (например, перебор всех GradeName)

Сравнение с незащищёнными копиями данных (тестовая среда, логи, кэш)

Side-channel атаки через время выполнения запроса или объём возвращаемых данных

Именно поэтому в серьёзных проектах DDM комбинируют с другими механизмами защиты.